Name: Yash B. Pagade
Roll.no: B 38

Software Testing Quality Assurance Lab
Practical Assignment 4
Experiment 4: Quality Assurance: Introduction to software quality assurance, Validation checks and Regression Testing.

Name: Yash B. Pagade
Div: B
Roll No.: 38
Batch: 1

Name: Yash B. Pagade
Roll.no: B 38

Quality Assurance

1. Software Quality Assurance

Software Quality Assurance (SQA) is a systematic process used to ensure that software is developed according to specified requirements, standards and procedures. It focuses on preventing defects, improving the development process and ensuring that the final software is reliable and meets user requirements.

2. Validation Checks

Validation is the process of checking whether the developed software satisfies the actual requirements of the user. It verifies the actual output against the expected output by executing the software. For example, checking whether valid login credentials successfully open the user dashboard.

3. Regression Testing

Regression testing is performed after changes or modifications are made to software. It ensures that the changes have not affected previously working features. Existing test cases are executed again to verify that the software continues to function correctly.

4. Importance

SQA, validation and regression testing help in detecting and preventing defects, maintaining software reliability and ensuring that the software continues to meet its specified requirements.

Name: Yash B. Pagade
Roll.no: B 38

Aim:

To demonstrate validation testing using TestNG assertions with Selenium WebDriver.

Description:

Validation testing is used to verify whether the actual result obtained from an application matches the expected result. In this practical, TestNG assertions are used to perform validation after carrying out an operation on a web application. The Assert.assertEquals() method compares the actual and expected values and determines whether the validation is successful.

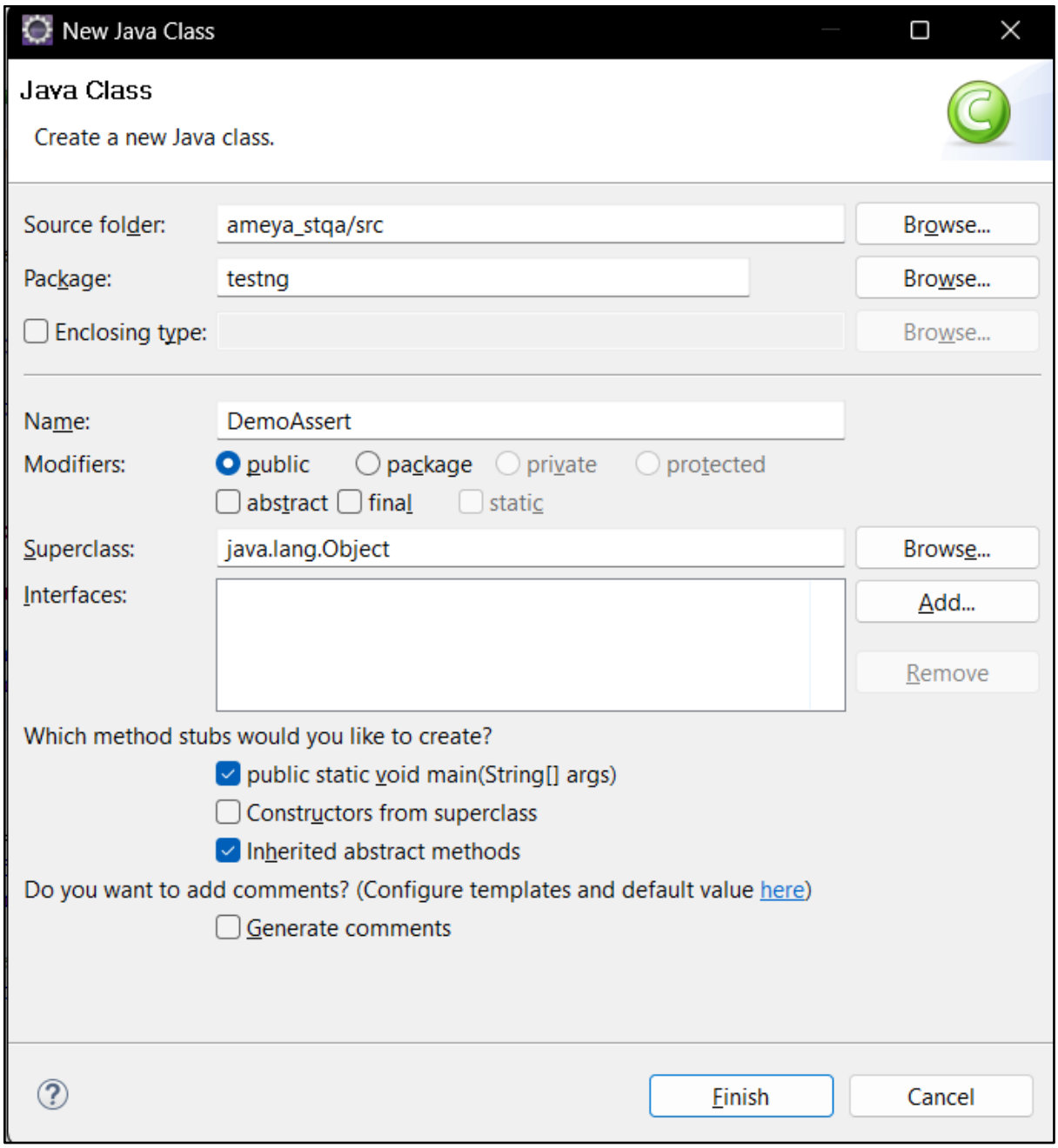
Implementation:

A TestNG class named DemoAssert was created in Eclipse. Selenium WebDriver was used to open the SauceDemo website and perform login using valid credentials. After successful login, the heading displayed on the Products page was obtained using Selenium. The actual heading was then compared with the expected value "Products" using Assert.assertEquals(). The test result was displayed by TestNG.

Name: Yash B. Pagade
Roll.no: B 38

Outputs:

Create a Class under the Testng package



Name: Yash B. Pagade
Roll.no: B 38

Insert the below code in the class file

Code:

DemoAssert.java

```
package testng;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class DemoAssert {

    WebDriver driver;

    @BeforeMethod
    public void setUp() {

        driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));

        driver.get("https://www.saucedemo.com/");
    }

    @Test
    public void validationTest() {

        // Enter username
        driver.findElement(By.id("user-name"))
              .sendKeys("standard_user");

        // Enter password
        driver.findElement(By.id("password"))
```

```
            .sendKeys("secret_sauce");

        // Click login
        driver.findElement(By.id("login-button"))
            .click();

        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        wait.until(driver ->
           driver.findElement(By.className("title")).isDisplayed()
        );

        // Validate page title
        String actualTitle =
              driver.findElement(By.className("title")).getText();

        String expectedTitle = "Products";

        Assert.assertEquals(actualTitle, expectedTitle);

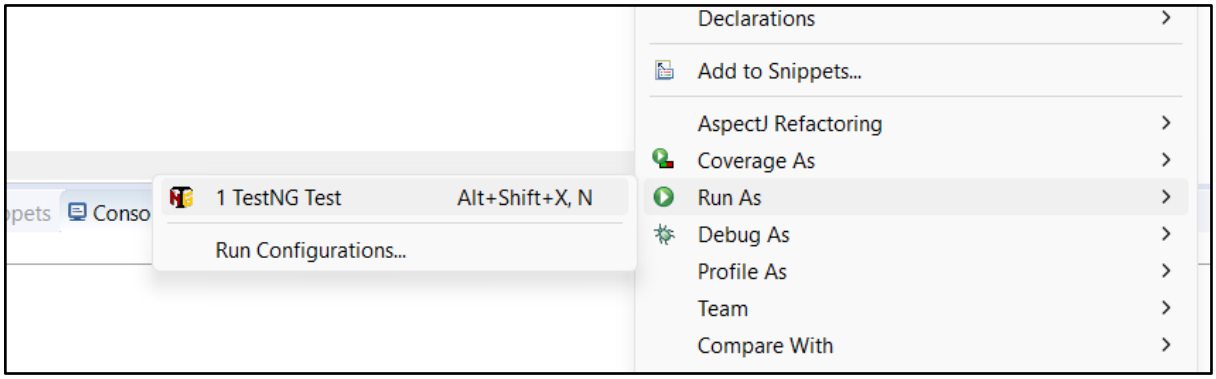
        System.out.println("Validation Successful");
        System.out.println("Expected: " + expectedTitle);
        System.out.println("Actual: " + actualTitle);
    }

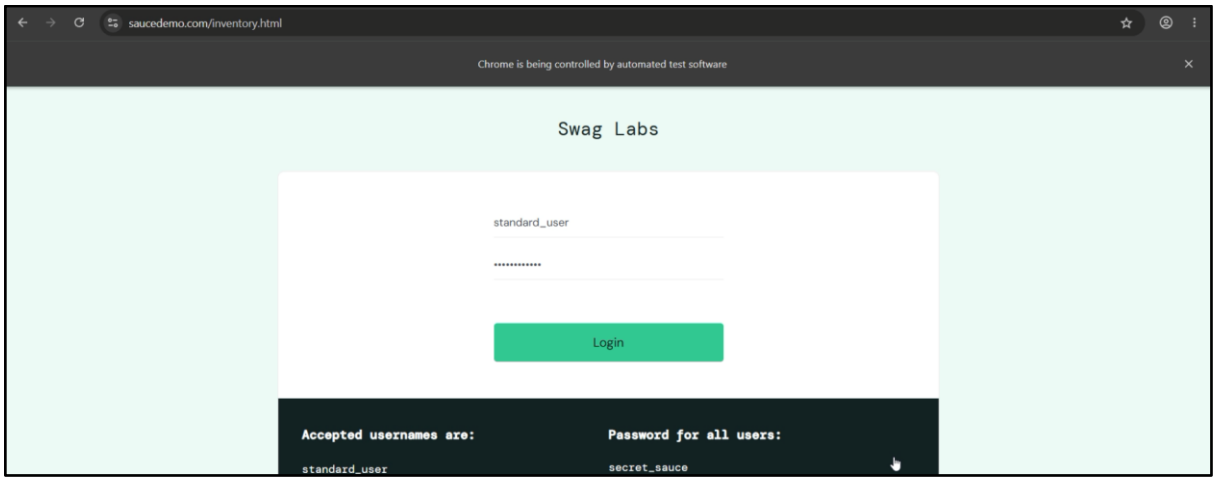
    @AfterMethod
    public void tearDown() {

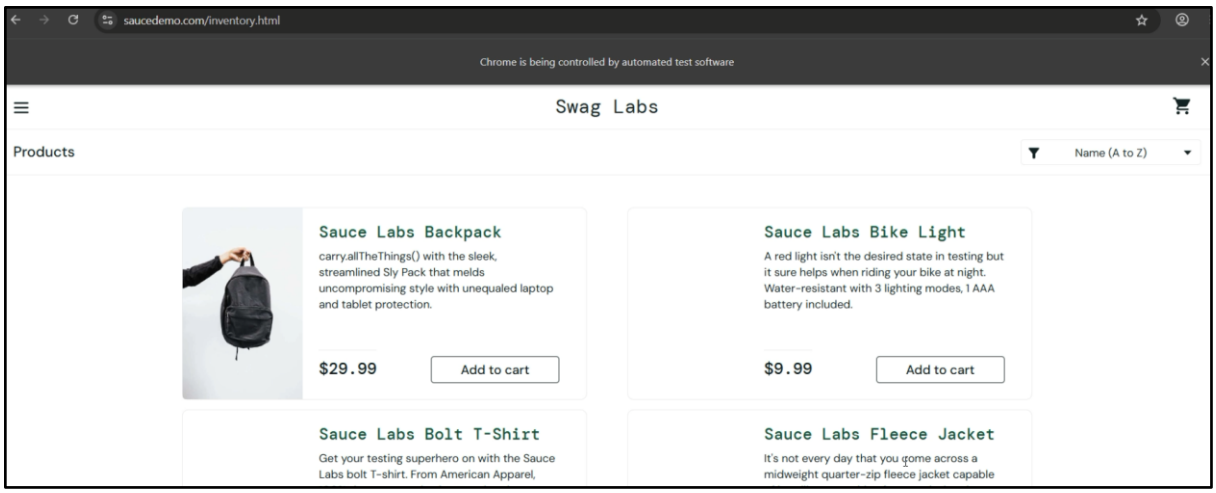
        driver.quit();
    }
}
```

Name: Yash B. Pagade
Roll.no: B 38

Run the class file as TestNG







Console Output

Name: Yash B. Pagade
Roll.no: B 38

```
Validation Successful
Expected: Products
Actual: Products
PASSED: testng.DemoAssert.validationTest

===============================================
    Default test
    Tests run: 1, Failures: 0, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
===============================================
```

The output shows that the login operation was performed successfully and the actual page heading obtained from the application was "Products", which matched the expected value. Therefore, the validation assertion was successful and TestNG reported the test as passed.

Result:

Validation testing was successfully demonstrated using Selenium WebDriver and TestNG assertions. The actual result of the application was compared with the expected result and the test was successfully validated.

Name: Yash B. Pagade
Roll.no: B 38

Lets Demonstrate a failed validation

Changed "**Products**" to "**Dashboard**" in expectedTitle

```
    String actualTitle =
            driver.findElement(By.className("title

    String expectedTitle = "Dashboard";

    Assert.assertEquals(actualTitle, expectedTitle

    System.out.println("Validation Successful");
```

Lets run and check the Console Output

```
FAILED: testng.DemoAssert.validationTest
java.lang.AssertionError: expected [Dashboard] but found [Products]
        at org.testng.Assert.fail(Assert.java:111)
        at org.testng.Assert.failNotEquals(Assert.java:1578)
        at org.testng.Assert.assertEqualsImpl(Assert.java:150)
        at org.testng.Assert.assertEquals(Assert.java:132)
        at org.testng.Assert.assertEquals(Assert.java:656)
        at org.testng.Assert.assertEquals(Assert.java:666)
        at testng.DemoAssert.validationTest(DemoAssert.java:58)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invok
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invok
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.i
        at java.base/java.lang.reflect.Method.invoke(Method.java:566)
```

```
===============================================
    Default test
    Tests run: 1, Failures: 1, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 1, Passes: 0, Failures: 1, Skips: 0
===============================================
```

This demonstrates that when the actual result doesn't match the expected result, **validation fails**

Name: Yash B. Pagade
Roll.no: B 38

Another Example

Code:

DemoAssert2.java

```
package testng;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class DemoAssert2 {

    WebDriver driver;

    @BeforeMethod
    public void setUp() {

        driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));

        driver.get("https://the-internet.herokuapp.com/add_remove_elements/");
    }

    @Test
    public void validationTest() {

        // Validate page title
        String actualTitle = driver.getTitle();
        String expectedTitle = "The Internet";

        Assert.assertEquals(actualTitle, expectedTitle);
```

```
        System.out.println("Page title validation successful");

        // Click Add Element
        driver.findElement(By.xpath("//button[text()='Add Element']")).click();

        // Validate Delete button is displayed
        boolean deleteButtonDisplayed =
                driver.findElement(By.className("added-manually")).isDisplayed();

        Assert.assertTrue(deleteButtonDisplayed);

        System.out.println("Add Element validation successful");

        // Click Delete
        driver.findElement(By.className("added-manually")).click();

        // Validate Delete button is removed
        int deleteButtonCount =
                driver.findElements(By.className("added-manually")).size();

        Assert.assertEquals(deleteButtonCount, 0);

        System.out.println("Delete Element validation successful");

        System.out.println("All validation checks passed");
    }

    @AfterMethod
    public void tearDown() {

        driver.quit();
    }
}
```

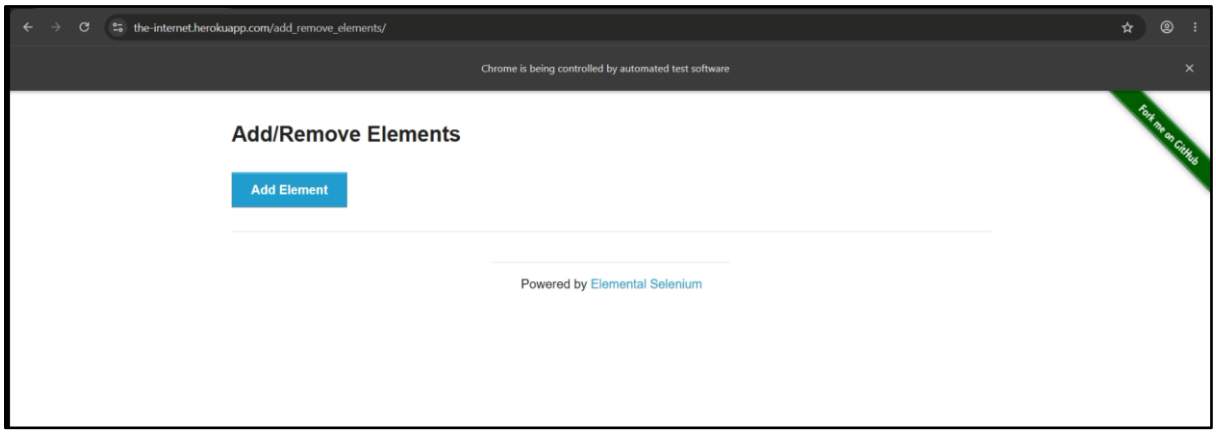
Name: Yash B. Pagade
Roll.no: B 38

Outputs:

```
Page title validation successful
Add Element validation successful
Delete Element validation successful
All validation checks passed
PASSED: testng.DemoAssert2.validationTest

===============================================
    Default test
    Tests run: 1, Failures: 0, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
===============================================
```



The output shows that the page title was successfully validated, the Delete button appeared after the Add Element operation, and the Delete button was successfully removed after the Delete operation. All validation checks were completed successfully and TestNG reported the test as passed.